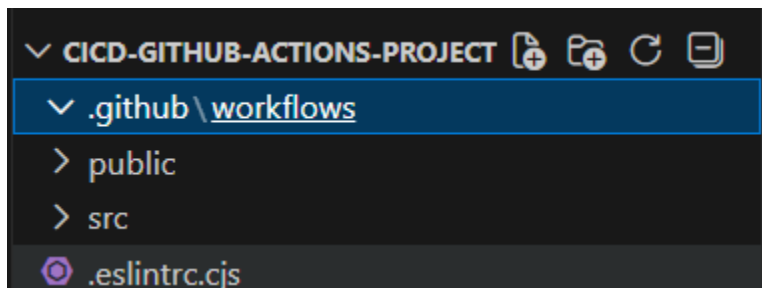


GITHUB ACTIONS END TO END DEPLOYMENT

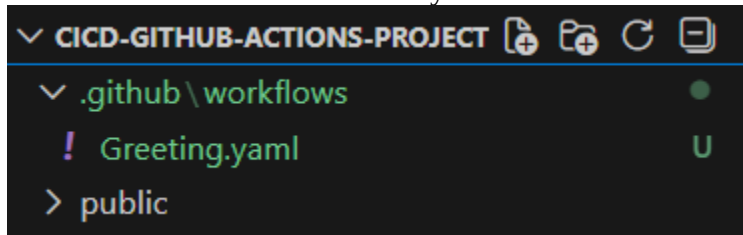
- Git repo: <https://github.com/akshu20791/cicd-github-actions-project.git>
- clone the Github repo in your local system
- Create a new Github repo and Push the code

1. First we have to print “Hello word” in Git action

- Create a folder → .github/workflows (in cloned repository)



- Under workflows folder create a yaml file named Greeting.yaml



Go to Extensions in Vscode and Install Github Extension

Now write in yaml file to run Hello World

Code:-

```
name: Greeting Workflow
on: push
jobs:
  greet:
    runs-on: ubuntu-latest
    steps:
      - name: Greet User
        run: echo "Hello, World!"
```

greet is the **job name** (GitHub Actions uses the word job)

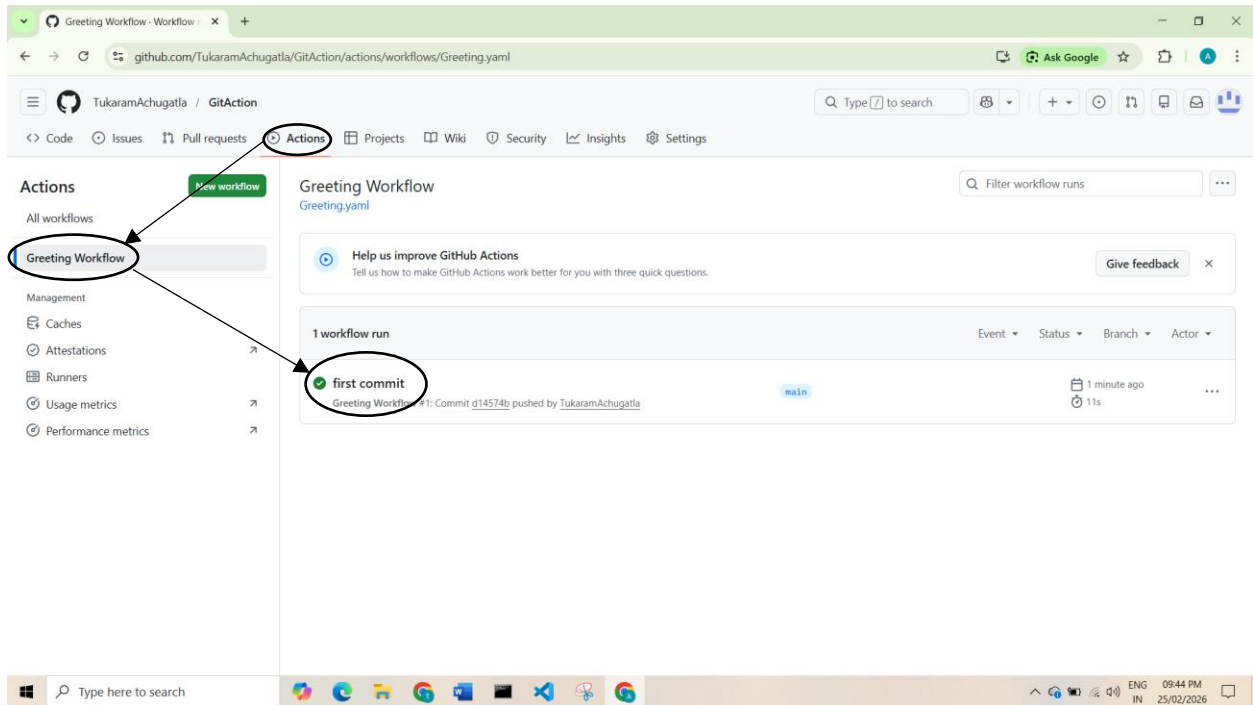
what is **runs-on: ubuntu-latest**?

This tells GitHub **which machine (runner) to use** to execute the job.

- `ubuntu-latest` → Runs on the latest Ubuntu Linux virtual machine.
- It is a **GitHub-hosted runner**.
- GitHub automatically provides this temporary server.

Now Add the code to Github repository and Push the code

Open GitHub → Click on Action → Greeting Workflow → first commit/greet → Greet User



The screenshot shows the GitHub Actions interface for a repository named 'TukaramAchugatla / GitAction'. The 'Actions' tab is selected, showing a workflow named 'Greeting Workflow' with a status of 'first commit #1' and a green checkmark indicating success. On the left sidebar, there are links for 'Summary', 'All jobs', 'Run details', 'Usage', and 'Workflow file'. The 'All jobs' section shows a job named 'greet' with a green checkmark. The main area displays the details of the 'greet' job, which succeeded 4 minutes ago. The job steps are: 'Set up job', 'Greet User', and 'Complete job'. The 'Greet User' step is expanded, showing two lines of output: '1 Run echo "Hello, World!"' and '4 Hello, World!'. The first line is circled in the image.

It will Trigger Automatically without any Button if you need button do the following steps(Optional)

Step 1: Information about trigger

Triggers are defined in : <https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows>

Step 2: Key to add Button

workflow_dispatch: can be used to manually get a button to run the github action workflow.

Ex:-

```
name: Greeting Workflow
on: [push,workflow_dispatch]
jobs:
```

2. We need to write the workflow where in the github runners first we will clone the github repo

- Create a yaml file in workflows
- There are 2 method to clone the git repo in git action

a. Difficult Method

```
name: Deploy dist
on: push
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        run: |
          echo "cloning repo.."
          echo "${{toJson(github)}}}"
          git clone https://github.com/${{github.repository}}.git
```

`${{ }}`= Expression Syntax

`toJson(github)`= converts the entire github object into JSON format

`github.repository`= your owner/repository-name

Now Add the code to Github repository and Push the code

Open GitHub → Click on Action → Getting Deploy dist → clone repo code → test

The screenshot shows the GitHub Actions interface for the repository 'TukaramAchugatla / GitAction'. The 'Actions' tab is selected, and the workflow 'clone repo code #1' is shown as successful. The 'test' job is selected, and its details are displayed. The 'Get Code' step is expanded, showing the following log output:

```
1 ▶ Run echo "cloning repo.."
228 cloning repo..
229 {
230   token: ***,
231   job: test,
232   ref: refs/heads/main,
233   sha: ca77e5f7526c78e017ceae9531f2b7d6b67577de,
234   repository: TukaramAchugatla/GitAction,
235   repository_owner: TukaramAchugatla,
236   repository_owner_id: 287963359,
237   repositoryUrl: git://github.com/TukaramAchugatla/GitAction.git,
238   run_id: 22406746128,
239   run_number: 1,
240   retention_days: 90,
241   run_attempt: 1,
242   artifact_cache_size_limit: 10,
243   repository_visibility: public,
```

b. Easy Method

Search for github action marketplace in google → click on Marketplace → search checkout → click on Checkout → Scroll Down see Usage → Copy - user: actions/checkout@v6

The image shows a two-step process for finding the GitHub Actions Checkout action. The first step is a Google search for "github actions marketplace". The search results show the GitHub Marketplace link, which is circled. The second step shows the GitHub Marketplace page with the search bar containing "Checkout", also circled. Below the search bar, the search results for "Checkout" are displayed, showing 87 results. The first result, "Checkout", is circled, and an arrow points from the search bar to it. The "Checkout" action is described as "Checkout a Git repository at a particular version". The "Sparse Checkout" action is also visible below it, described as "Sparse checkout a git repository, especially to speedup mono-repositories clone time".

Google

github actions marketplace

AI Mode All Shopping Images Videos Short videos Forums More Tools

GitHub
https://github.com › marketplace › type=actions
Marketplace · GitHub

Actions · TruffleHog OSS · Metrics embed · yq - portable yaml processor · Super-Linter · Rebuild
Armbian and Kernel · Gosec Security Checker · Checkout · OpenCommit — ...

Enhance your workflow with extensions

Tools from the community and partners to simplify tasks and automate processes

Checkout

Search results for "Checkout"

87 results

Filter: All By: All creators Sort: Popularity

Checkout Checkout a Git repository at a particular version Action

Sparse Checkout Sparse checkout a git repository, especially to speedup mono-repositories clone time Action

What's new

Please refer to the [release page](#) for the latest release notes.

Usage

```
- uses: actions/checkout@v6
  with:
    # Repository name with owner. For example, actions/checkout
    # Default: ${{ github.repository }}
    repository: ''
```

Code :

```
name: Deploy dist
on: [push, workflow_dispatch]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
```

What is **uses: actions/checkout@v6** ?

Use a pre-built GitHub Action called checkout (version 6)

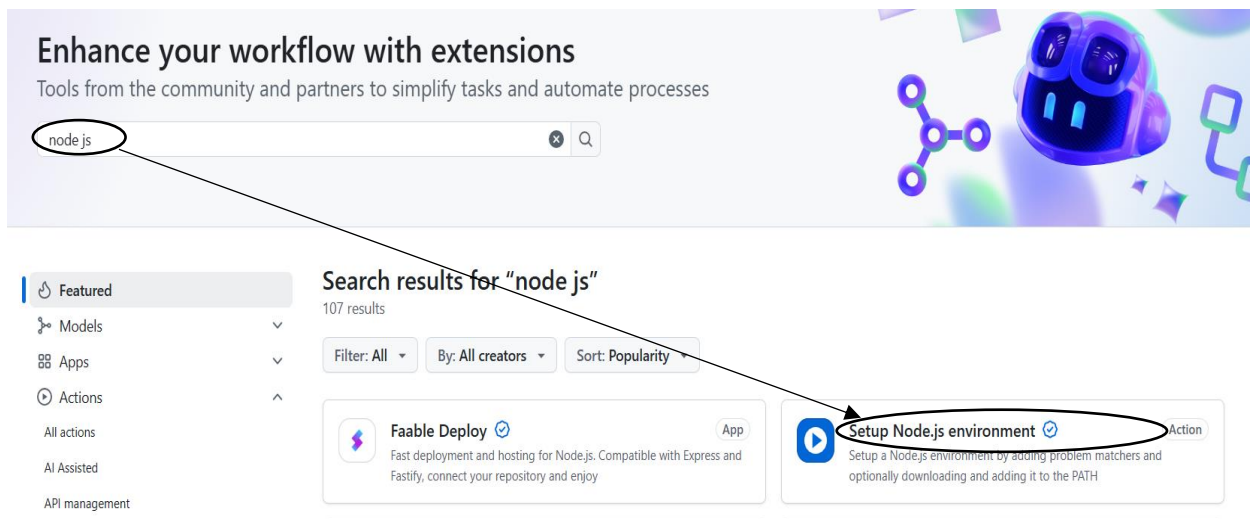
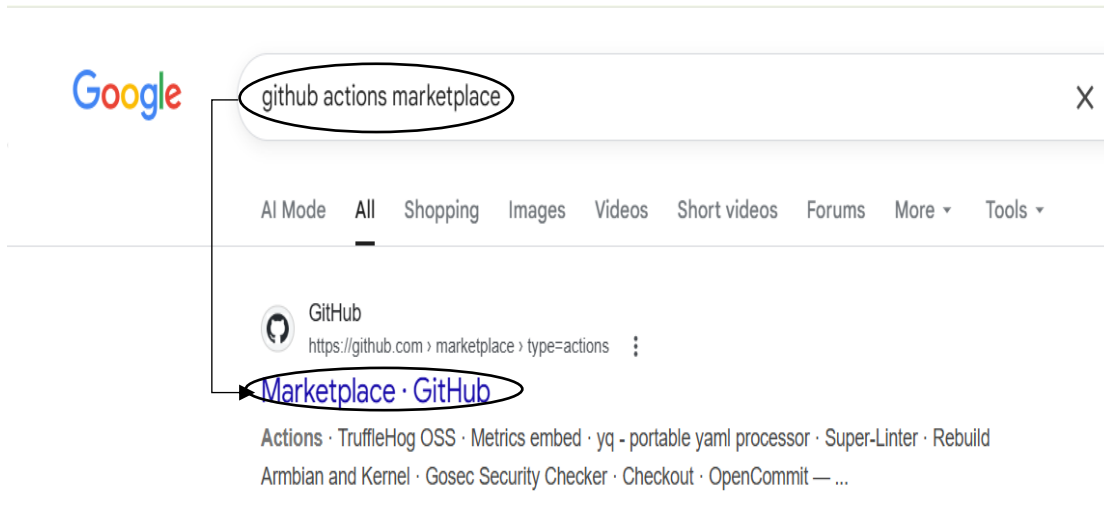
This action downloads (clones) your repository code into the GitHub runner

- As soon as we push the code to GitHub, the workflow will automatically execute.

```
✓ Get Code
28 hint: will change to "main" in Git 3.0. To configure the initial branch name
29 hint: to use in all of your new repositories, which will suppress this warning,
30 hint: call:
31 hint:
32 hint:         git config --global init.defaultBranch <name>
33 hint:
34 hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
35 hint: 'development'. The just-created branch can be renamed via this command:
36 hint:
37 hint:         git branch -m <name>
38 hint:
39 hint: Disable this message with "git config set advice.defaultBranchName false"
40 Initialized empty Git repository in /home/runner/work/GitAction/GitAction/.git/
41 /usr/bin/git remote add origin https://github.com/TukaramAchugatla/GitAction
42 ► Disabling automatic garbage collection
44 ▼ Setting up auth
```

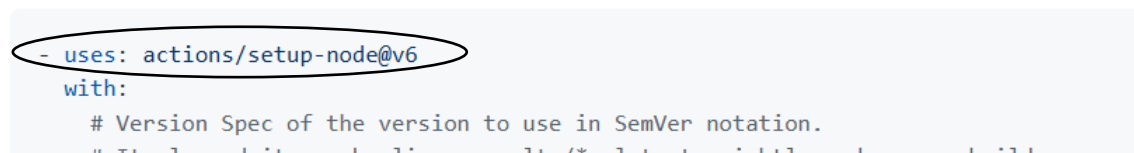
3. Now we need to install npm in the github runner

Search for github action marketplace in google → click on Marketplace → search node js → click on Setup Node.js environment → Scroll Down see Usage → Copy - user: actions/setup-node@v6



Usage

See [action.yml](#)



```
Code:-
name: Deploy dist
on: [push,workflow_dispatch]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
```

What is **uses: actions/setup-node@v6**?
It is an official GitHub Action
It installs **Node.js** in the GitHub runner

test

succeeded 3 minutes ago in 5s

- >  Set up job
- >  Get Code
- >  Install Node
- >  Post Install Node
- >  Post Get Code
- >  Complete job

4. Now let's print version of Node.js

Code :-

```
name: React Version
on: [push,workflow_dispatch]
jobs:
  ReactVersion:
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Version of Node .js
        run: node -v
```

The screenshot shows the GitHub Actions interface for a workflow named 'ReactVersion' by user 'TukaramAchugatla'. The workflow is in a 'Succeeded' state, having completed 4 minutes ago. The job 'ReactVersion' is highlighted in the left sidebar. The main panel shows the steps of the job: 'Set up job', 'Get Code', 'Install Node', and 'Version of Node .js'. The 'Version of Node .js' step is expanded, showing the command 'Run node -v' and the output 'v20.20.0'.

TukaramAchugatla / **GitAction**

<> Code Issues Pull requests **Actions** Projects Wiki

← React Version

✓ **check Version #3**

Summary

All jobs

✓ **ReactVersion**

Run details

Usage

Workflow file

ReactVersion
succeeded 4 minutes ago in 5s

- > ✓ Set up job
- > ✓ Get Code
- > ✓ Install Node
- ✓ ✓ **Version of Node .js**
 - 1 ▶ Run node -v
 - 4 v20.20.0

5. Now let's install Dependence in React Project

npm install= Installs dependencies from **package.json**

npm ci= Installs exactly what is in **package-lock.json**

Code:-

```
name: React Version
on: [push,workflow_dispatch]
jobs:
  ReactVersion:
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Version of Node .js
        run: node -v
      - name: Install Dependencies
        run: npm ci
```

What does **run: npm ci** mean?

It runs the command npm ci inside the GitHub runner

It installs all project dependencies

It installs them using the package-lock.json file

The screenshot shows the GitHub Actions interface for a repository named 'TukaramAchugatla / GitAction'. The 'Actions' tab is selected, and the workflow 'ReactDependencies' is shown as 'succeeded now in 10s'. The workflow steps are listed as follows:

- > Set up job
- > Get Code
- > Install Node
- > Version of Node .js
- > **Install Dependencies** (highlighted with a red circle)

The 'Install Dependencies' step is expanded, showing the command 'Run npm ci'.

Fail Test Case and Skip Workflow

6. Now let's Run Test

Code:-

```
name: React Run Test
on: [push,workflow_dispatch]
jobs:
  Run_Test:
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Version of Node .js
        run: node -v
      - name: Install Dependencies
        run: npm ci
      - name: Run Test
        run: npm test
```

What does **run: npm test** mean?

It runs the test script of your project

It checks whether your application is working correctly

It runs inside the GitHub runner

The screenshot shows the GitHub Actions interface for a workflow named 'React Run Test'. The 'Run Test #1' job is highlighted with a green checkmark, indicating it has succeeded. The 'Run Test' step within this job is also highlighted with a green checkmark and a red circle. The job summary shows it succeeded 2 minutes ago in 11s. The list of steps includes: Set up job, Get Code, Install Node, Version of Node .js, Install Dependencies, Run Test, Post Install Node, Post Get Code, and Complete job.

React Run Test

Run Test #1

Summary

All jobs

Run details

Usage

Workflow file

Run_Test

succeeded 2 minutes ago in 11s

- > ✓ Set up job
- > ✓ Get Code
- > ✓ Install Node
- > ✓ Version of Node .js
- > ✓ Install Dependencies
- > ✓ Run Test
- > ✓ Post Install Node
- > ✓ Post Get Code
- > ✓ Complete job

7. Now let's do fail test case and it will show in red colour

Open src → open App.text.jsx → on line number 21 add .not before .to (expect(h2Element).not.toBeInTheDocument());

Add the code to github and push the code

Code :-

```
name: React Run Test
on: [push,workflow_dispatch]
jobs:
  Run_Test:
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Version of Node .js
        run: node -v
      - name: Install Dependencies
        run: npm ci
      - name: Run Test
        run: npm test
```

The screenshot shows the GitHub Actions interface for a workflow named 'React Run Test'. The specific run is titled 'Run fail TestCase #2' and is marked as failed. The left sidebar shows the 'Run_Test' job selected. The main area displays a list of steps in the workflow:

- Set up job (successful)
- Get Code (successful)
- Install Node (successful)
- Version of Node .js (successful)
- Install Dependencies (successful)
- Run_Test (failed, indicated by a red 'x' icon)
- Post Install Node (successful)
- Post Get Code (successful)

The 'Run_Test' job failed 4 minutes ago in 31s.

- **Note:-**

If I add this comment in the commit message, Git should not trigger the workflow. So we can use: `git commit -m "added commit [skip ci]"`.

Paraller and Sequential Jobs

8. Now let's run the tests, build, and deploy the project.

Code:-

name: Build the React App

on: [push,workflow_dispatch]

jobs:

Run_Test:

runs-on: ubuntu-latest

steps:

- name: Get Code
uses: actions/checkout@v6
- name: Install Node
uses: actions/setup-node@v6
with:
node-version: '20'
- name: Version of Node .js
run: node -v
- name: Install Dependences
run: npm ci
- name: Run Test
run: npm test

Deploy:

runs-on: ubuntu-latest

steps:

- name: Get Code
uses: actions/checkout@v6
- name: Install Node
uses: actions/setup-node@v6
with:
node-version: '20'
- name: Install Dependences
run: npm ci
- name: Build Project
run: npm run build
- name: Deploy
run: echo "Deploying to production server..."

Run_Test **and** Deploy **are stages** in your workflow

Run_Test stage/job → installs, then runs npm test

Deploy stage/job → installs, builds, then deploys

← Build the React App

✓ Build project #3

Summary

All jobs

- ✓ Run_Test
- ✓ Deploy

Run details

- Usage
- Workflow file

Triggered via push now

 **TukaramAchugatla** pushed  `7dc665b` [main](#)

Test_Build.yaml

on: push

- | | |
|------------|-----|
| ✓ Run_Test | 10s |
| ✓ Deploy | 11s |

Run_Test

succeeded 1 minute ago in 10s

- > ✓ Set up job
- > ✓ Get Code
- > ✓ Install Node
- > ✓ Version of Node .js
- > ✓ Install Dependencies
- > ✓ **Run Test**
- > ✓ Post Install Node
- > ✓ Post Get Code
- > ✓ Complete job

Deploy

succeeded 1 minute ago in 11s

- > ✓ Set up job
- > ✓ Get Code
- > ✓ Install Node
- > ✓ Install Dependencies
- > ✓ **Build Project**
- > ✓ Deploy
- > ✓ Post Install Node
- > ✓ Post Get Code
- > ✓ Complete job

*****Note:-** It will run the Run_Test and Deploy in parallel. If we want to run Deploy after Run_Test then we need to add (needs: Run_Test) in the Deploy Job

Code :-

```
name: Build the React App
on: [push,workflow_dispatch]
jobs:
  Run_Test:
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Version of Node .js
        run: node -v
      - name: Install Dependences
        run: npm ci
      - name: Run Test
        run: npm test
  Deploy:
    needs: Run_Test
    runs-on: ubuntu-latest
    steps:
      - name: Get Code
        uses: actions/checkout@v6
      - name: Install Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
      - name: Install Dependences
        run: npm ci
      - name: Build Project
        run: npm run build
      - name: Deploy
        run: echo "Deploying to production server..."
```

TukaramAchugatla / GitAction

<> Code Issues Pull requests **Actions** Projects Wiki Security Insights Settings

← Build the React App

Build project after Run_Test #4

Summary

All jobs

- Run_Test
- Deploy

Run details

- Usage
- Workflow file

Triggered via push now

TukaramAchugatla pushed to 88265a4 main

Status: In progress

Total duration: —

Test_Build.yaml

on: push

Run_Test 19s → Deploy 12s

Now If the test fails, then it should not deploy

- Open src → open App.text.jsx → on line number 21 add .not before .to (expect(h2Element).not.toBeInTheDocument();)
- Add the code to github and push the code

Code:-

name: Build the React App

on: [push,workflow_dispatch]

jobs:

Run_Test:

runs-on: ubuntu-latest

steps:

- name: Get Code

uses: actions/checkout@v6

- name: Install Node

uses: actions/setup-node@v6

with:

node-version: '20'

- name: Version of Node .js

run: node -v

- name: Install Dependences

run: npm ci

- name: Run Test

run: npm test

Deploy:

needs: Run_Test

runs-on: ubuntu-latest

steps:

- name: Get Code
uses: actions/checkout@v6
- name: Install Node
uses: actions/setup-node@v6
with:
node-version: '20'
- name: Install Dependencies
run: npm ci
- name: Build Project
run: npm run build
- name: Deploy
run: echo "Deploying to production server..."

← Build the React App

✖ Build project after Run_Test but test should get fail #5

Summary

All jobs

- ✖ Run_Test
- Deploy

Run details

- Usage
- 📄 Workflow file

Triggered via push now

TukaramAchugatla pushed [4b58c3d](#) [main](#)

Status: **Failure** Total duration: **16s**

Test_Build.yaml
on: push

✖ Run_Test 12s → ○ Deploy 0s

Cache Dependency

In Run_Test and Deploy, we are installing the same dependencies, which takes 7 seconds. To reduce that time, we can use dependency caching.

Let's go to Marketplace → search cache → scroll down for Node -npm → copy – user: actions/cache@v5 → copy key → copy path

The screenshot shows a Google search interface. The search bar contains the text "github actions marketplace". Below the search bar, the "All" tab is selected. The first search result is from GitHub, titled "Marketplace · GitHub", with the URL "https://github.com › marketplace › type=actions". A red arrow points from the search bar to the "Marketplace · GitHub" link. Below the link, a list of actions is visible, including "Actions · TruffleHog OSS · Metrics embed · yq - portable yaml processor · Super-Linter · Rebuild Armbian and Kernel · Gosec Security Checker · Checkout · OpenCommit — ...".

Enhance your workflow with extensions

Tools from the community and partners to simplify tasks and automate processes

cache



Featured

Models

Apps

Actions

All actions

AI Assisted

API management

Backup Utilities

Chat

Code quality

Search results for "cache"

492 results

Filter: All

By: All creators

Sort: Popularity



Buildless



App

Lightning fast build caching



Cache



Action

Cache artifacts like dependencies and build outputs to improve workflow execution time

Implementation Examples

Every programming language and framework has its

See [Examples](#) for a list of `actions/cache` implementations

- [Bun](#)
- [C# - NuGet](#)
- [Clojure - Lein Deps](#)
- [D - DUB](#)
- [Deno](#)
- [Elixir - Mix](#)
- [Go - Modules](#)
- [Haskell - Cabal](#)
- [Haskell - Stack](#)
- [Java - Gradle](#)
- [Java - Maven](#)
- [Node - npm](#)
- [Node - Lerna](#)
- [Node - Yarn](#)
- [OCaml/Reason - esy](#)

Scroll Down

PWSH shell

```
- name: Get npm cache directory
  id: npm-cache-dir
  shell: pwsh
  run: echo "dir=$(npm config get cache)" >> ${env:GITHUB_OUTPUT}
```

Get npm cache directory step can then be used with `actions/cache` as shown below

Copy

```
uses: actions/cache@v5
id: npm-cache # use this to check for `cache-hit` ==> if: steps.npm-cache.outputs.cache-hit != 'true'
with:
  path: ${{ steps.npm-cache-dir.outputs.dir }}
  key: ${{ runner.os }}-node-{{ hashFiles('**/package-lock.json') }}
  restore-keys: |
    ${{ runner.os }}-node-
```

Code:-

name: Cache Dependences
on: [push,workflow_dispatch]
jobs:

Run_Test:

runs-on: ubuntu-latest
steps:

```
- name: Get Code
  uses: actions/checkout@v6
- name: Install Node
  uses: actions/setup-node@v6
  with:
    node-version: '20'
- name: Cache Node Modules
  uses: actions/cache@v5
  with:
    path: ~/.npm
    key: ${{ runner.os }}-node-{{ hashFiles('**/package-lock.json') }}
- name: Install Dependencies
  run: npm ci
- name: Run Test
  run: npm test
```

Deploy:

```
needs: Run_Test
runs-on: ubuntu-latest
steps:
- name: Get Code
  uses: actions/checkout@v6
- name: Install Node
```

It will Generate a unique key based on the OS and the hash of the package-lock.json file

```
uses: actions/setup-node@v6
with:
  node-version: '20'
- name: Cache Node Modules
  uses: actions/cache@v5
  with:
    path: ~/.npm
    key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
- name: Install Dependencies
  run: npm ci
- name: Build Project
  run: npm run build
- name: Deploy
  run: echo "Deploying to production server..."
```

Before::

Run_Test succeeded 1 hour ago in 19s			Q Search logs	🔄 ⚙️
>	✔️ Set up job			1s
>	✔️ Get Code			1s
>	✔️ Install Node			6s
>	✔️ Version of Node.js			0s
>	✔️ Install Dependencies			7s
>	✔️ Run Test			1s
Deploy succeeded 1 hour ago in 12s			Q Search logs	🔄 ⚙️
>	✔️ Set up job			1s
>	✔️ Get Code			0s
>	✔️ Install Node			1s
>	✔️ Install Dependencies			6s
>	✔️ Build Project			1s
>	✔️ Deploy			0s

After adding cache Dependency::

The image shows two screenshots of GitHub Actions workflow logs. The top screenshot is for a job named 'Run_Test' which succeeded 1 minute ago. It lists steps: 'Set up job' (0s), 'Get Code' (1s), 'Install Node' (1s), and 'Cache Dependencies' (2s). The 'Cache Dependencies' step is expanded, showing logs for 'Run actions/cache@v5' with a cache hit, download speeds, and cache size. Below this, the 'Install Dependencies' step is circled in red. The bottom screenshot is for a job named 'Deploy' which succeeded 2 minutes ago. It lists steps: 'Set up job' (1s), 'Get Code' (0s), 'Install Node' (3s), and 'Cache Dependencies' (1s). The 'Cache Dependencies' step is expanded, showing logs for 'Run actions/cache@v5' with a cache hit, download speeds, and cache size. Below this, the 'Install Dependencies' step is circled in red. The 'Deploy' job also includes a 'Build Project' step (2s).

Run_Test
succeeded 1 minute ago in 15s

Search logs

- > Set up job 0s
- > Get Code 1s
- > Install Node 1s
- ▼ Cache Dependencies 2s
 - 1 ▶ Run actions/cache@v5
 - 9 Cache hit for: Linux-node-b0d420026e14e92147f47155980fed8854c66c1e56a052f74b3d4d686c35fcd1
 - 10 Received 9079365 of 21662277 (41.9%), 8.7 MBs/sec
 - 11 Received 21662277 of 21662277 (100.0%), 18.4 MBs/sec
 - 12 Cache Size: ~21 MB (21662277 B)
 - 13 /usr/bin/tar -xf /home/runner/work/_temp/9602305e-669e-4223-b519-24c2581c1c9d/cache.tzst -P -C /home/runner/work/GitAction/GitAction --use-compress-program unzstd
 - 14 Cache restored successfully
 - 15 Cache restored from key: Linux-node-b0d420026e14e92147f47155980fed8854c66c1e56a052f74b3d4d686c35fcd1
- > Install Dependencies 5s

Deploy
succeeded 2 minutes ago in 15s

Search logs

- > Set up job 1s
- > Get Code 0s
- > Install Node 3s
- ▼ Cache Dependencies 1s
 - 1 ▶ Run actions/cache@v5
 - 9 Cache hit for: Linux-node-b0d420026e14e92147f47155980fed8854c66c1e56a052f74b3d4d686c35fcd1
 - 10 Received 21662277 of 21662277 (100.0%), 129.1 MBs/sec
 - 11 Cache Size: ~21 MB (21662277 B)
 - 12 /usr/bin/tar -xf /home/runner/work/_temp/cc4d24b8-cc1d-4b0f-8248-d960c096f0fd/cache.tzst -P -C /home/runner/work/GitAction/GitAction --use-compress-program unzstd
 - 13 Cache restored successfully
 - 14 Cache restored from key: Linux-node-b0d420026e14e92147f47155980fed8854c66c1e56a052f74b3d4d686c35fcd1
- > Install Dependencies 4s
- > Build Project 2s

Filter On Event Push

Now, the workflow should trigger only when code is pushed to the main branch or feature.
Explain:-

on:

push:

branches:

- main
- feature/**

paths-ignore:

- README.md
- '.github/workflows/**'

Example: -

This means the workflow will trigger when you push to branches:

- main:- main branch only
- feature/** :- Any branch starting with feature(like → feature/login, feature/payment, etc)

The workflow will **NOT run if only these files are changed**

paths-ignore:

- README.md
- '.github/workflows/**'

- Only README.md → ❌ Workflow will NOT trigger
- Only workflow file change → ❌ Workflow will NOT trigger
- Code file change (like `App.js`) → ✅ Workflow will trigger

So, Lets check first without making any changes in App.js file will be it trigger or Not

Code :-

name: Branches

on: [push,workflow_dispatch]

on:

push:

branches:

- main
- feature/**

paths-ignore:

- README.md
- '.github/workflows/**'

workflow_dispatch:

jobs:

Run_Test:

runs-on: ubuntu-latest

steps:

- name: Get Code
uses: actions/checkout@v6
- name: Install Node
uses: actions/setup-node@v6
with:
node-version: '20'
- name: Cache Dependencies
uses: actions/cache@v5
with:
path: ~/.npm
key: \${{ runner.os }}-node-\${{ hashFiles('**/package-lock.json') }}

- name: Install Dependencies
run: npm ci
- name: Run Test
run: npm test

Deploy:

needs: Run_Test

runs-on: ubuntu-latest

steps:

- name: Get Code
uses: actions/checkout@v6
- name: Install Node
uses: actions/setup-node@v6
with:
node-version: '20'
- name: Cache Dependencies
uses: actions/cache@v5
with:
path: ~/.npm
key: \${{ runner.os }}-node-\${{ hashFiles('**/package-lock.json') }}
- name: Install Dependencies
run: npm ci
- name: Build Project
run: npm run build
- name: Deploy
run: echo "Deploying to production server..."

The screenshot shows the GitHub Actions interface for the repository 'TukaramAchugatla / GitAction'. The top navigation bar includes links for Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings. The 'Actions' section is active, showing a list of workflows on the left and a detailed view of 'Gitbranch.yaml' on the right. The workflow view indicates '0 workflow runs' and states 'This workflow has a workflow_dispatch event trigger.' A blue play button icon with a checkmark is visible. A red oval highlights the text 'This workflow has no runs yet.' at the bottom right of the interface.

This will not trigger if no changes are made to any files, except the README file and the .github/workflows directory.

So, Let's make changes in App.jsx file to h2.tag ("This is React App")

Before:-

```
src > App.jsx > App
1  import './App.css'
2
3  function App() {
4
5      return (
6          <>
7              <h1>Github Actions</h1>
8              <h2>Complete course with 2 projects</h2>
9          </>
10     )
11 }
12
13 export default App
14
```

After:-

```
src > App.jsx > App
1  import './App.css'
2
3  function App() {
4
5      return (
6          <>
7              <h1>Github Actions</h1>
8              <h2>This is React App</h2>
9          </>
10     )
11 }
12
13 export default App
14
```

Now add App.jsx Code to Github And Push the code

Now the git is Trigger

The screenshot shows the GitHub Actions interface. On the left, there's a sidebar with 'Branches' and 'Filter on Branches #1'. Below that, a 'Summary' tab is selected, showing 'All jobs' and a list of jobs including 'Run_Test'. The main area displays the workflow details for 'Run_Test'. It shows the workflow was triggered via push now, pushed by 'TukaramAchugatla' to the 'main' branch. The status is 'In progress'. Below this, the 'Gitbranch.yaml' file is shown with the trigger 'on: push'. The workflow diagram shows a sequence of jobs: 'Run_Test' (6s) followed by 'Deploy'.

Upload and Download artifact

Explain

What is an Artifact?

An **artifact** is a file or folder generated during a workflow that we want to Save temporarily, Share between jobs, Download later.

Examples:

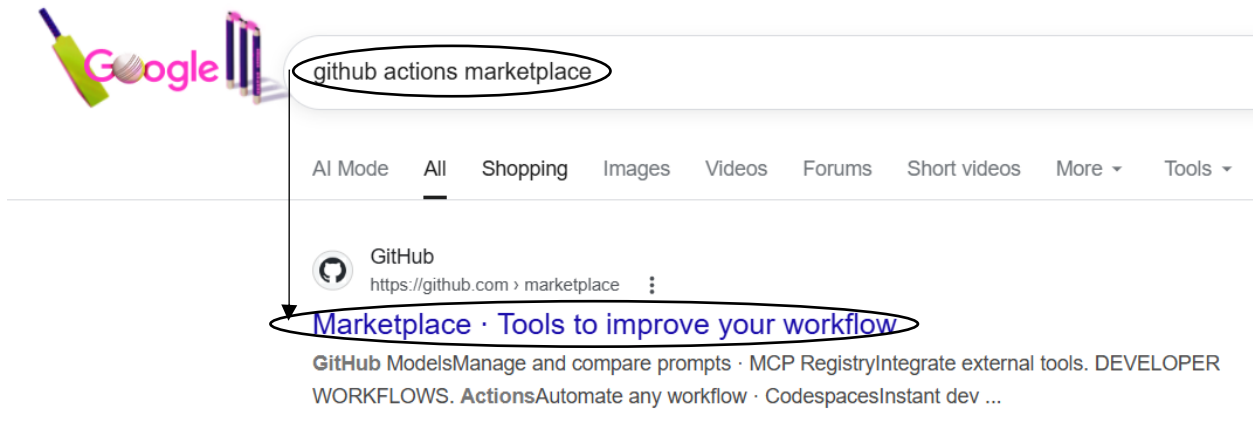
1. Test reports
2. JAR/WAR files

Why Do We Use Artifacts?

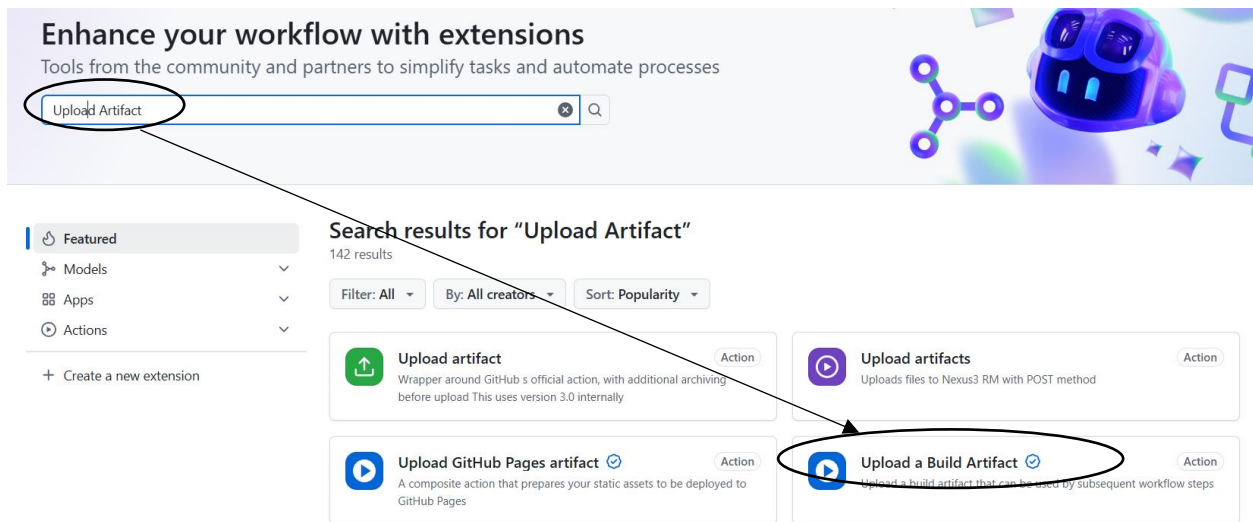
To transfer build output or important files from one job to another job in a workflow.

Real Example (CI/CD Flow)

- Job 1 – Build
 - Install dependencies
 - Run tests
 - Build project
- Job 2 – Deploy
 - Download artifact
 - Deploy to server

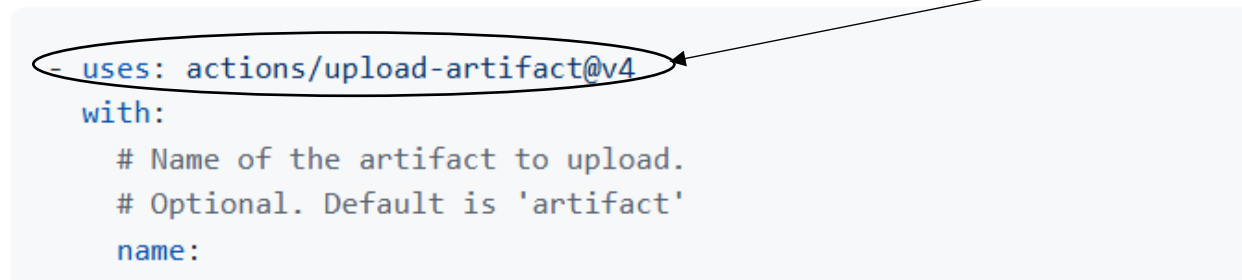


This is to upload Artifact



Usage

Inputs

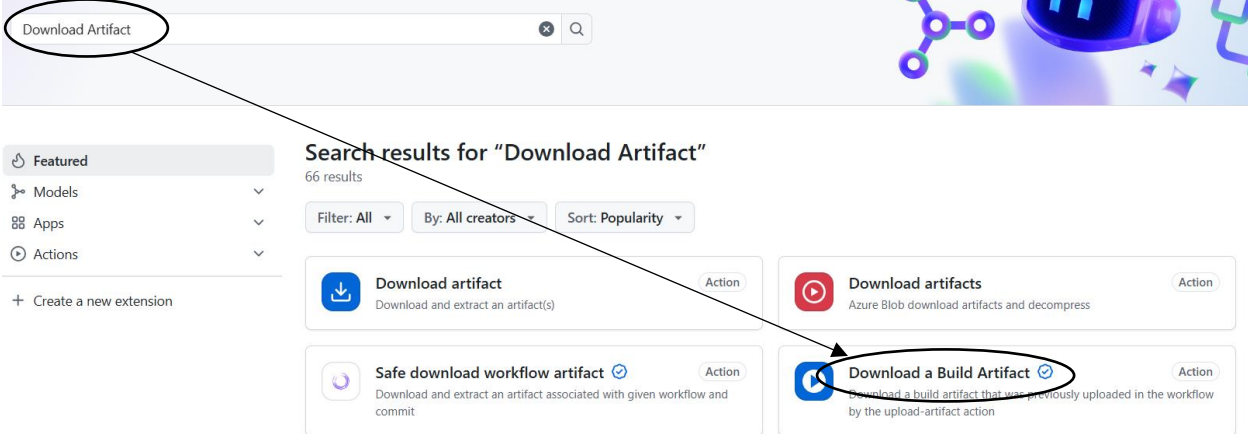


Copy

This Is to Download Artifact

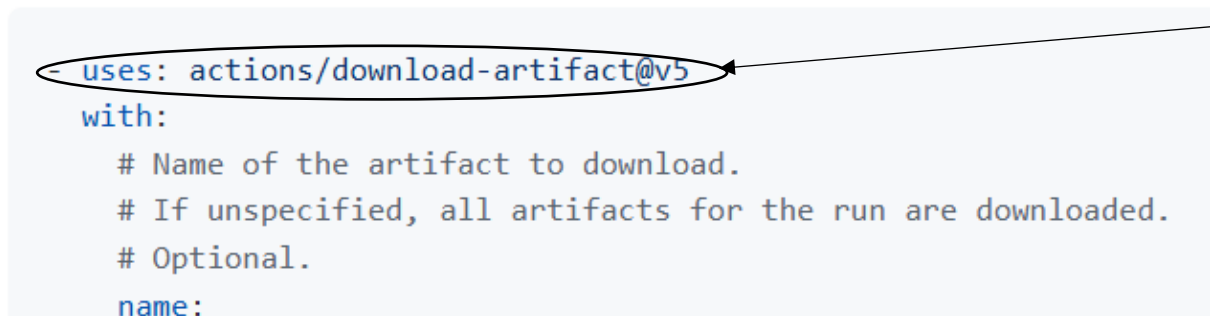
Enhance your workflow with extensions

Tools from the community and partners to simplify tasks and automate processes



Usage

Inputs



Code:

```
name: Branches
# on: [push,workflow_dispatch]
on:
  push:
    branches:
      - main
      - feature/**
  paths-ignore:
    - README.md
    - '.github/workflows/**'
  workflow_dispatch:
jobs:
  Run_Test:
    runs-on: ubuntu-latest
```

steps:

- name: Get Code
uses: actions/checkout@v6
- name: Install Node
uses: actions/setup-node@v6
with:
node-version: '20'
- name: Cache Dependencies
uses: actions/cache@v5
with:
path: ~/.npm
key: \${{ runner.os }}-node-\${{ hashFiles('**/package-lock.json') }}
- name: Install Dependencies
run: npm ci
- name: Run Test
run: npm test

Build:

- needs: Run_Test
runs-on: ubuntu-latest
steps:
- name: Get Code
uses: actions/checkout@v6
 - name: Install Node
uses: actions/setup-node@v6
with:
node-version: '20'
 - name: Cache Dependencies
uses: actions/cache@v5
with:
path: ~/.npm
key: \${{ runner.os }}-node-\${{ hashFiles('**/package-lock.json') }}
 - name: Install Dependencies
run: npm ci
 - name: Build Project
run: npm run build
 - name: Upload Artifact
uses: actions/upload-artifact@v4
with:
name: dist-files
path: dist

Here Is the code for
Upload Artifact

Deploy:

- needs: Build
runs-on: ubuntu-latest
steps:
- name: Download Artifact
uses: actions/download-artifact@v5

Here is the code
For Download
Artifact

with:
name: dist-files
path: dist
- name: Get Code
run: echo "Getting code..."

Note:-

Now, when we add code to GitHub and push it, the workflow will not trigger because of **paths-ignore**. We have specified that if changes are made only to those files, the workflow should not run.

Now open github → click on your repo → click on Action →

The screenshot displays the GitHub Actions interface for a repository named 'TukaramAchugatla / GitAction'. The 'Actions' tab is selected, showing a list of workflows on the left and the details of the 'Update and Download Artifact' workflow on the right. A callout box points to the 'Run workflow' button, stating 'Click here To Trigger'. Another callout box points to the workflow run status, stating 'It is Not Trigger Yet'. Below this, the workflow configuration for 'Uploadartifact.yaml' is shown, triggered on 'workflow_dispatch'. The workflow steps are 'Run_Test' (14s), 'Build' (11s), and 'Deploy' (5s). A callout box points to the 'Build' step, stating 'Build has been Completed'. Below the workflow steps, the 'Artifacts' section is visible, showing a single artifact named 'dist-files' with a size of 46.7 KB and a SHA256 digest. A callout box points to the download icon for the artifact, stating 'You can Download Artifact which is nothing but a zip file'. A final callout box points to the artifact ID, stating 'Where This Artifact id consist of Html, Css, JavaScript files'.

Click here To Trigger

It is Not Trigger Yet

Update and Download Artifact

Uploadartifact.yaml

1 workflow run

This workflow has a workflow_dispatch event trigger.

Update and Download Artifact #1: Manually run by TukaramAchugatla

main

Run workflow

Use workflow from

Branch: main

Run workflow

Uploadartifact.yaml

on: workflow_dispatch

Run_Test 14s

Build 11s

Deploy 5s

Build has been Completed

Artifacts

Produced during runtime

Name	Size	Digest
dist-files	46.7 KB	sha256:c6e8dbaccd458f1186aee5964aba1fa0b84a0ec7b09...

You can Download Artifact which is nothing but a zip file

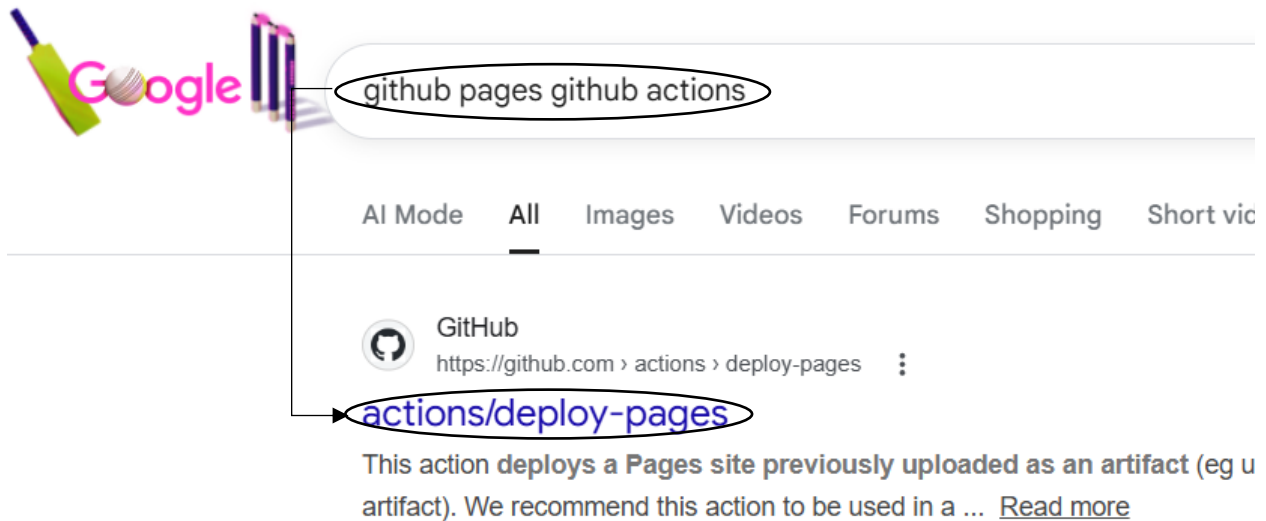
Where This Artifact id consist of Html, Css, JavaScript files

Deploy to GitHub pages using Github action

Now we are going to deploy a React application using GitHub Actions. We can deploy it through GitHub Pages, which allows us to host React or other web applications. Using GitHub Actions, we automate the deployment of the React application to GitHub Pages.

Create one file with DeployReactApp.yaml in .github/workflows

Search for github pages github action



Steps to Deploy the React Application Using GitHub Pages

1. Scroll down to the “Usage” section in the GitHub Pages documentation.
2. Copy the code from the “Deploy” section and paste it into your DeployReactApp.yaml file.
3. In the build job, **modify the upload artifact step** as shown below
 - name: Upload Artifact
 - uses: actions/upload-pages-artifact@v3
 - with:
 - name: github-pages
 - path: dist
4. Next, open the package.json file and add the following homepage URL
"homepage: https://TukaramAchugatla.github.io/GitAction"

```
{ } package.json > { } scripts
1  {
2    "name": "react-demo",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "homepage": "https://TukaramAchugatla.github.io/GitAction",
```

This is my Github
user Name

This is my Github
Repository Name

5. Next, open the vite.config.js file and add your repository name in the base property as shown below:

base: '/GitAction/',

```
6   plugins: [react()],
7   test: {
8     environment: 'jsdom',
9     setupFiles: ['./vitest.setup.js'],
10  },
11  base: '/GitAction/'
12  })
13
```

Github
Repository Name

Code :-

name: Deploy React to GitHub Pages

on:

push:

branches:

- main
- feature/**

paths-ignore:

- README.md
- '.github/workflows/**'

workflow_dispatch:

jobs:

Run_Test:

runs-on: ubuntu-latest

steps:

- name: Get Code
uses: actions/checkout@v6
- name: Install Node
uses: actions/setup-node@v6
with:
node-version: '20'
- name: Cache Dependencies
uses: actions/cache@v5
with:
path: ~/.npm
key: \${{ runner.os }}-node-\${{ hashFiles('**/package-lock.json') }}
- name: Install Dependencies
run: npm ci
- name: Run Test
run: npm test

Build:

needs: Run_Test

runs-on: ubuntu-latest

steps:

- name: Get Code
 - uses: actions/checkout@v6
- name: Install Node
 - uses: actions/setup-node@v6
 - with:
 - node-version: '20'
- name: Cache Dependencies
 - uses: actions/cache@v5
 - with:
 - path: ~/.npm
 - key: \${{ runner.os }}-node-\${{ hashFiles('**/package-lock.json') }}
- name: Install Dependencies
 - run: npm ci
- name: Build Project
 - run: npm run build
- name: Upload Artifact
 - uses: actions/upload-pages-artifact@v3
 - with:
 - name: github-pages
 - path: dist

Deploy:

needs: Build

runs-on: ubuntu-latest

Grant GITHUB_TOKEN the permissions required to make a Pages deployment permissions:

pages: write # to deploy to Pages

id-token: write # to verify the deployment originates from an appropriate source

Deploy to the github-pages environment

environment:

name: github-pages

url: \${{ steps.deployment.outputs.page_url }}

steps:

- name: Deploy to GitHub Pages
 - id: deployment
 - uses: actions/deploy-pages@v4
 - with:
 - token: \${{ secrets.GITHUB_TOKEN }}

<> Code Issues Pull requests Actions Projects Wiki Security Insights Settings

← Deploy React to GitHub Pages

Added New Workflow #1

Summary

All jobs

Run_Test
Build
Deploy

Run details
Usage
Workflow file

Triggered via push 1 minute ago

TukaramAchugatla pushed · b48ddb3 · main

Status: Failure

Total duration: 45s

Artifacts: 1

DeployReactApp.yaml

on: push

Run_Test (16s) → Build (13s) → Deploy (6s)

Click Here

If the workflow fails as shown above, go to the Settings tab

<> Code Issues Pull requests Actions Projects Wiki Security Insights Settings

GitHub Pages source saved.

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Codespaces

Pages

GitHub Pages

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.

Build and deployment

Source

Deploy from a branch

GitHub Actions

Best for using frameworks and customizing your build process

✓ Deploy from a branch

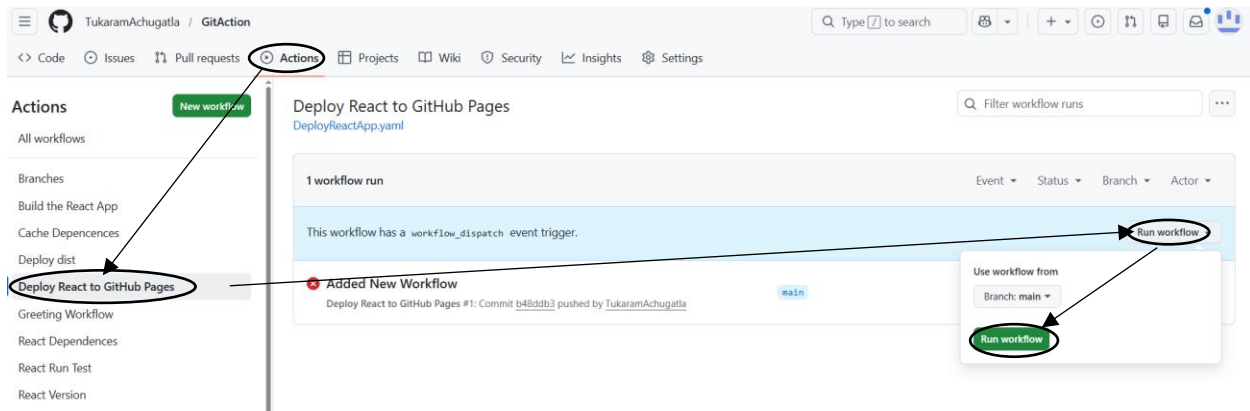
Classic Pages experience

Learn how to add a Jekyll theme to your site.

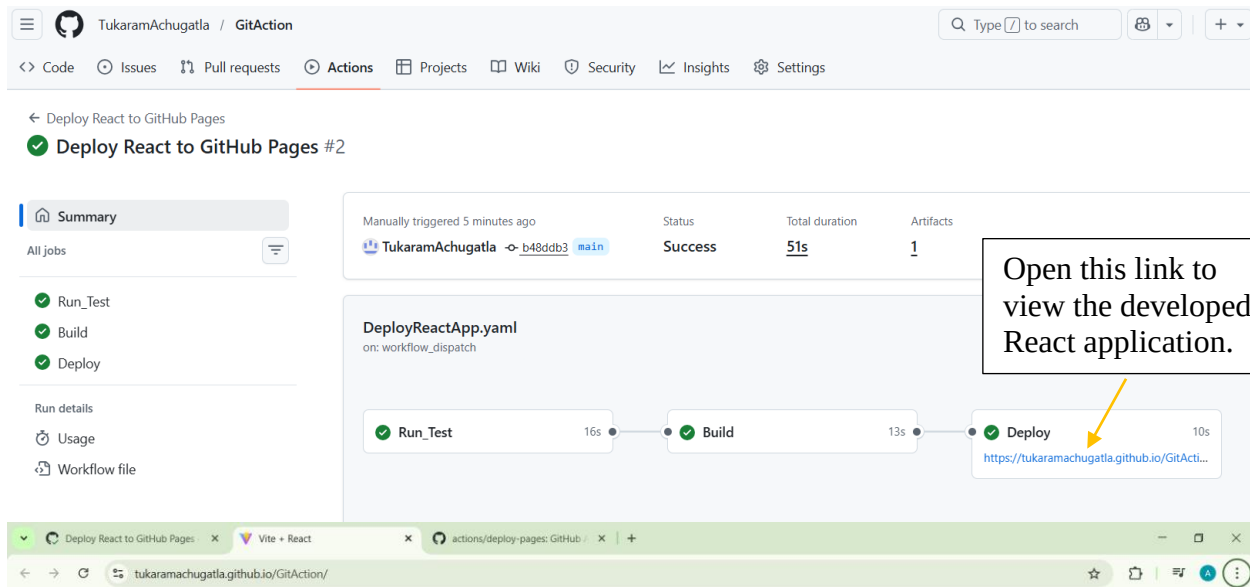
Custom domain

Custom domains allow you to serve your site from a domain other than tukaramachugatla.github.io. Learn more about configuring custom domains.

Save Remove



Now its Done



Github Actions

This is React App



Integrate Sonar Cloud with Git action

Case Study:-

If any developer pushes their code to a feature branch and raises a Pull Request (PR), we first check the code quality using SonarCloud integration before merging.

If the code quality is not good, I will not approve the PR and will inform the developer about the issues.

Once the developer fixes the issues and meets the required quality standards, I will review and approve the code.

Steps to Integrate SonarCloud with GitHub

◆ SonarCloud Account Setup

1. Search for **SonarCloud** on Google.
2. Log in using your **GitHub account**.
3. Click on **Authorize SonarQubeCloud**.
4. Select the **Free organization plan** and create the organization.
5. Click on **“Get Started with GitHub.”**
6. Choose **All repositories**, then click **Install & Authorize**.
7. Enter your GitHub password and click **Confirm**.
8. Scroll down, select the **Free organization plan**, and click **Create**.
9. Click on **Analyze New Project**.
10. Select the project you want to integrate (for example, **GitAction**) and click **Set Up**.
11. Click on **Previous Version**, then click **Create Project**.

◆ Configure SonarCloud in GitHub

12. Open the **SonarQube Cloud Dashboard**.
13. Click on **Administration → Analysis Method**.
14. Enable **Automatic Analysis**.
15. Click on your **repository name**.
16. Copy the **Name** and **Value** fields and save them in Notepad.

◆ Add SonarCloud Secrets in GitHub

17. Go to your GitHub repository → **Settings**.
18. Click on **Secrets and Variables → Actions**.
19. Click on **New repository secret**.
20. Paste the copied **Name** and **Value** fields.
21. Click on **Add Secret**.

◆ Add SonarCloud Workflow File

22. Go back to the SonarCloud dashboard and scroll down to **Other CI**.
23. Copy the provided GitHub Actions YAML code.
24. In VS Code, create a new file named `.github/workflows/sonarIssues.yaml`
25. Paste the copied YAML code into this file.

◆ Add sonar-project.properties File

26. In the SonarCloud dashboard, scroll down and copy the **sonar-project.properties** content.
27. Create a new file in your project root (outside `.github/workflows`) named `sonar-project.properties`
28. Paste the copied content into this file.

◆ Run the Workflow

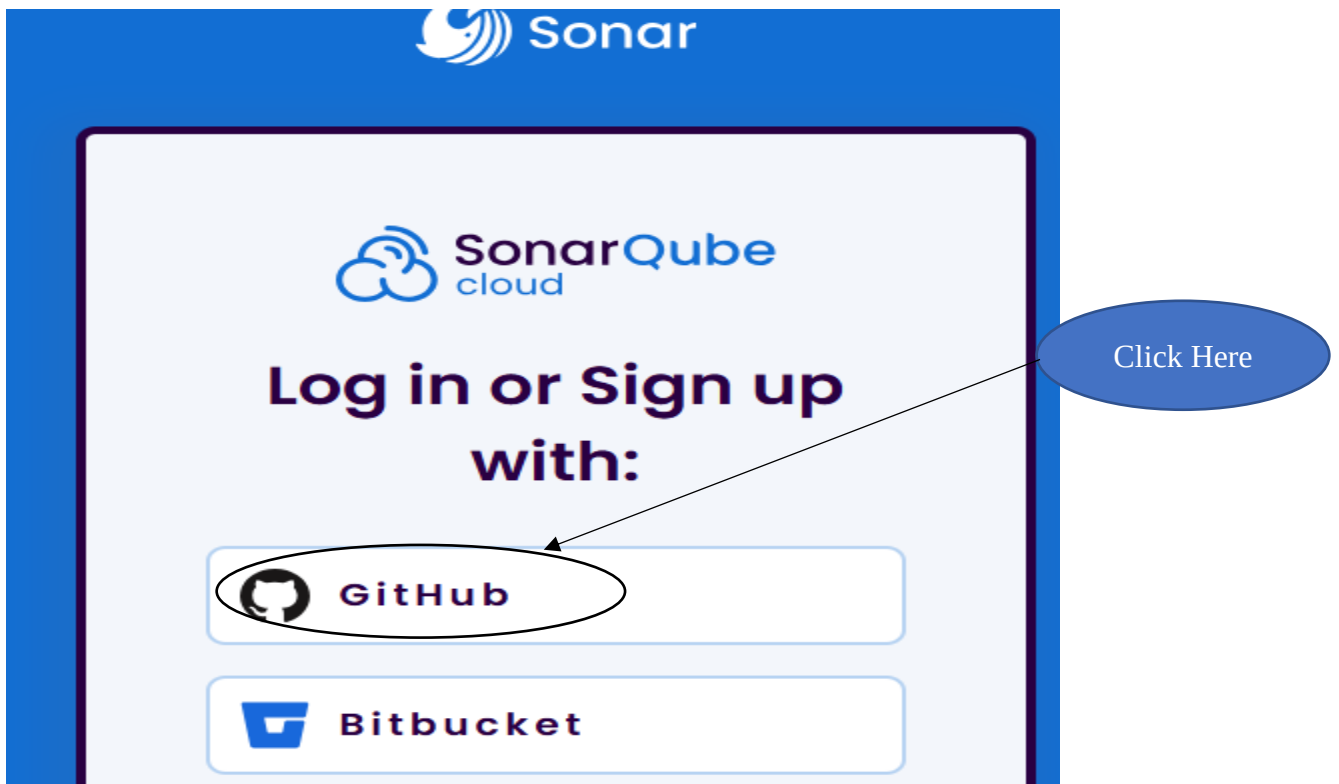
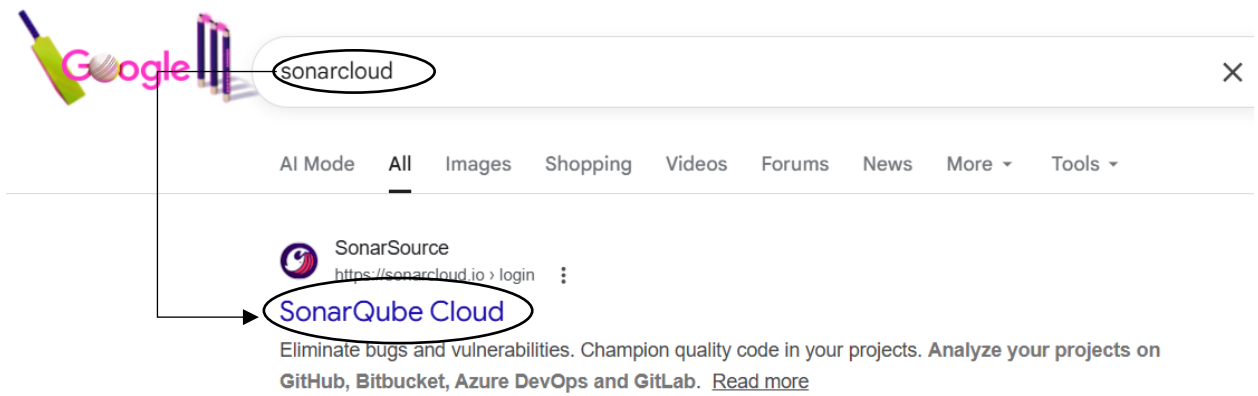
29. Push the code to GitHub.
30. Go to the **Actions tab** and verify that the SonarCloud workflow runs successfully

◆ Create New Branch and Raise PR

31. Create a new branch named `newh5tag`
32. Add the following tag in `App.jsx` `<h5>with SonarCloud</h5>`
33. Push the branch to GitHub.
34. Raise a **Pull Request (PR)** from `newh5tag` to `main`.
35. Wait for the SonarQubeCloud check to complete.
36. If the Quality Gate passes, click **Merge**.

◆ Final Result

37. After merging, SonarQubeCloud should pass successfully.
38. Finally, open the **SonarQube Cloud dashboard** to view the complete analysis report.





SonarQubeCloud by **Sonar** would like permission to:

 Verify your GitHub identity (TukaramAchugatla)

 Know which resources you can access

 Act on your behalf
[? Learn more](#)

Resources on your account

 **Email addresses** (read)
View your email addresses

[Learn more about SonarQubeCloud](#)

Cancel

Authorize SonarQubeCloud

Authorizing will redirect to



Click Here

Welcome to SonarQube Cloud
Ready to improve your code quality and security?

 **Get started with GitHub**

Install & Authorize SonarQubeCloud

Install & Authorize on your personal account TukaramAchugatla



for these repositories:

☒ **All repositories**

This applies to all current and future repositories owned by the resource owner. Also includes public repositories (read-only).

☐ **Only select repositories**

Select at least one repository. Also includes public repositories (read-only).

with these permissions:

- ✓ **Read** access to code and metadata
- ✓ **Read** and **write** access to checks, commit statuses, pull requests, and security events

User permissions

Installing and authorizing SonarQubeCloud immediately grants these permissions on your account: **TukaramAchugatla**.

- ✓ **Read** access to email addresses

Install & Authorize

[Cancel](#)

Next: you'll be redirected to <https://sonarcloud.io/callback>

Confirm access



Signed in as
@TukaramAchugatla

Password

[Forgot password?](#)

Confirm

Enter Your Github
Password

Tip: You are entering [sudo mode](#). After you've performed a sudo-protected action, you'll only be asked to re-authenticate again after a few hours of inactivity.



[My Projects](#) [My Issues](#) [Explore](#)

Create an organization

Organizations enable your team to collaborate across many projects.

1 Import organization details

Import **TukaramAchugatla** into a SonarQube Cloud organization ×

Name *

TukaramAchugatla

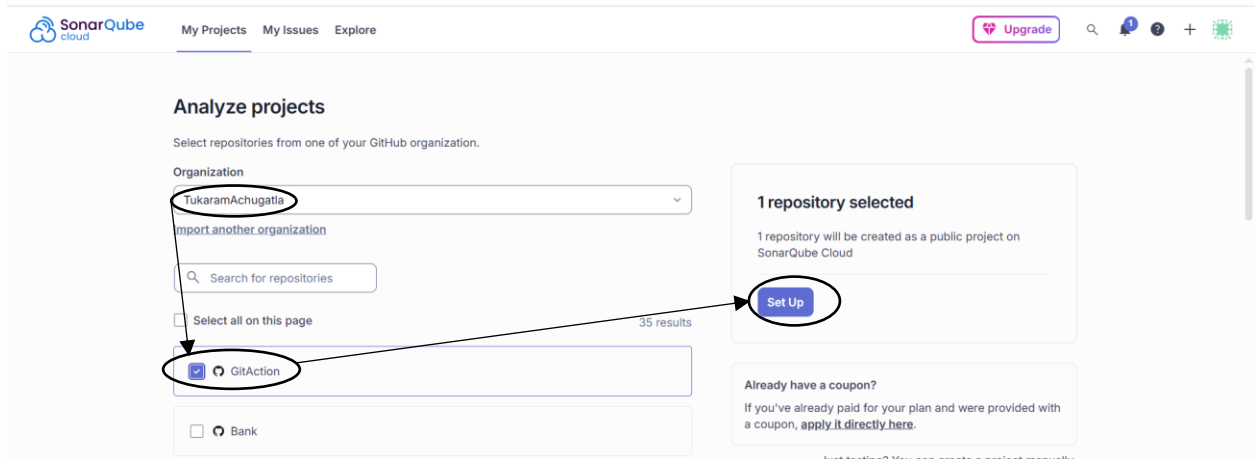
Up to 255 characters

Key * ⓘ

tukaramachugatla

Use only lowercase letters, numbers, or hyphens. Must start and end with a letter or number. Up to 255 characters.

[> Add additional info](#)



Set up new code for project

The new code definition sets the criteria for what's considered new code, allowing you to focus on the most recent changes in your project.

This setting allows project administrators to customize the new code definition for their project. [Learn more in documentation](#) 

Your organization's new code definition is not set

Set a new code definition for your organisation to use it by default for all new projects

[TukaramAchugatla - Administration - New Code](#)

New code definition for this project:

☒ Previous version

Any code that has changed since the previous version is considered new code. Recommended for **projects following regular versions or releases**.

☐ Number of days

Any code that has changed in the last x days is considered new code. If no action is taken on a new issue after x days, this issue will become part of the overall code. Recommended for **projects following continuous delivery**.

[Back](#)

[Create project](#)

Open the SonarQube Cloud Dashboard.

The screenshot shows the SonarQube Cloud interface. The browser address bar displays `sonarcloud.io/project/configuration/AutoScan?id=TukaramAchugatla_GitAction`. The left sidebar contains a menu with 'Administration' circled in black. A blue arrow points from a 'Click Here' callout bubble to this 'Administration' link. The main content area is titled 'Project onboarding' and features a gear icon. It includes the text 'We are analyzing your project.' and a list of bullet points: 'You will get a first analysis of your default branch and the 5 most recently active Pull Requests.', 'Each new push on your default branch will trigger a new analysis automatically.', and 'Each new push to a Pull Request will also trigger an analysis.' Below this, it says 'Check these useful links while you wait:' followed by a link 'Learn about the SonarQube Cloud Core Concepts while you wait'.

This screenshot shows the 'Analysis method' configuration page. The left sidebar has 'Administration' circled in black, with a line connecting it to the 'Analysis method' link, which is also circled in black. Below 'Analysis method' are links for 'General settings' and 'New code'. The main content area is titled 'Analysis method' and contains a section 'Automatic Analysis' with a toggle switch that is currently turned off. A green callout bubble with the text 'Enable this(Means turn it off)' has an arrow pointing to this toggle switch. Below the toggle is a section 'Set up analysis via other methods' with two options: 'With GitHub Actions' (circled in black) and 'With CircleCI'.

Now go to your Github repositories

The screenshot shows the SonarQube 'Project onboarding' page for a project named 'GitAction'. The left sidebar contains a navigation menu with options like 'Quality gate', 'Pull Requests', 'Branches', 'Code', 'Project Information', and 'Administration'. The main content area is titled 'Analyze a project with a GitHub Action' and includes two steps: '1 Create a GitHub Secret' and '2 Create or update a build file'. Step 1 provides instructions to create a new secret in a GitHub repository with the name 'SONAR_TOKEN' and a specific value. Arrows from a text box point to these fields. Step 2 offers various project type options like 'JS/TS & Web', 'Maven', 'Gradle', etc.

Copy Paste this Name and Value field in Notepad

Click On Setting

The screenshot shows the GitHub repository settings page for 'TukaramAchugatla / GitAction'. The top navigation bar includes links for 'Code', 'Issues', 'Pull requests', 'Actions', 'Projects', 'Wiki', 'Security', 'Insights', and 'Settings'. The 'Settings' link is circled. On the left, a sidebar menu lists settings categories: 'Security', 'Advanced Security', 'Deploy keys', 'Secrets and variables', 'Actions', 'Codespaces', and 'Dependabot'. The 'Secrets and variables' option is circled, and an arrow points to the 'Actions' option below it. The main content area shows the 'Social preview' section with instructions on how to upload a repository image.

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets

Variables

Environment secrets

This environment has no secrets.

Manage environment secrets

Repository secrets

This repository has no secrets.

New repository secret

Click On New repository secret

Actions secrets / New secret

Name *

SONAR_TOKEN

Paste the Name Here From Notepad

Secret *

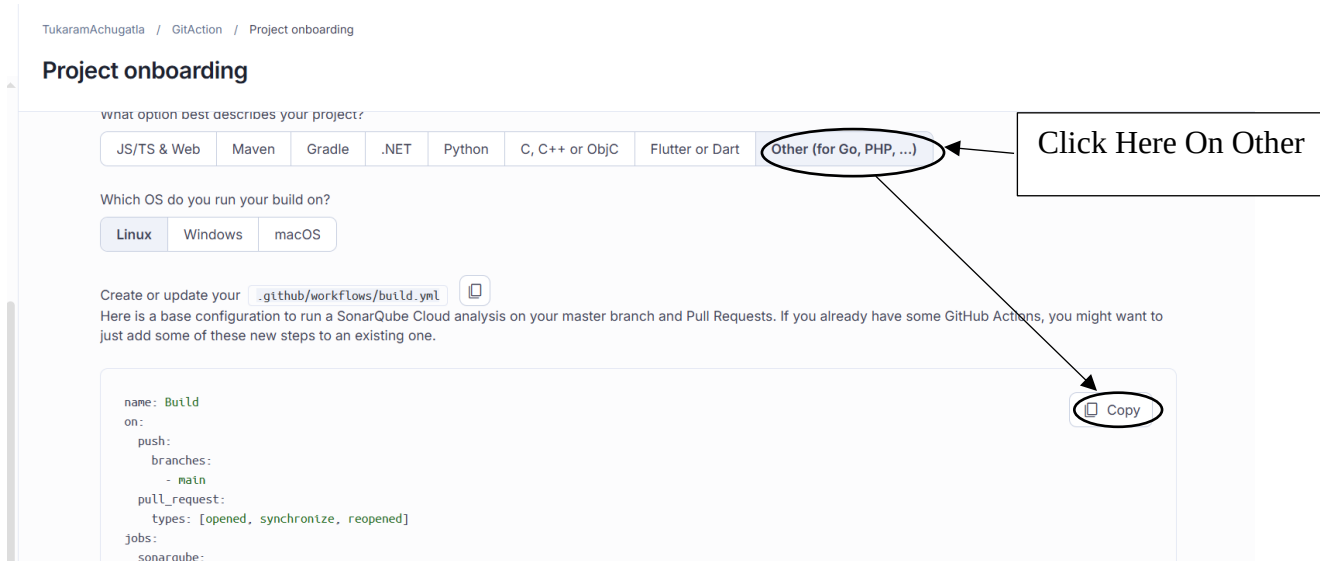
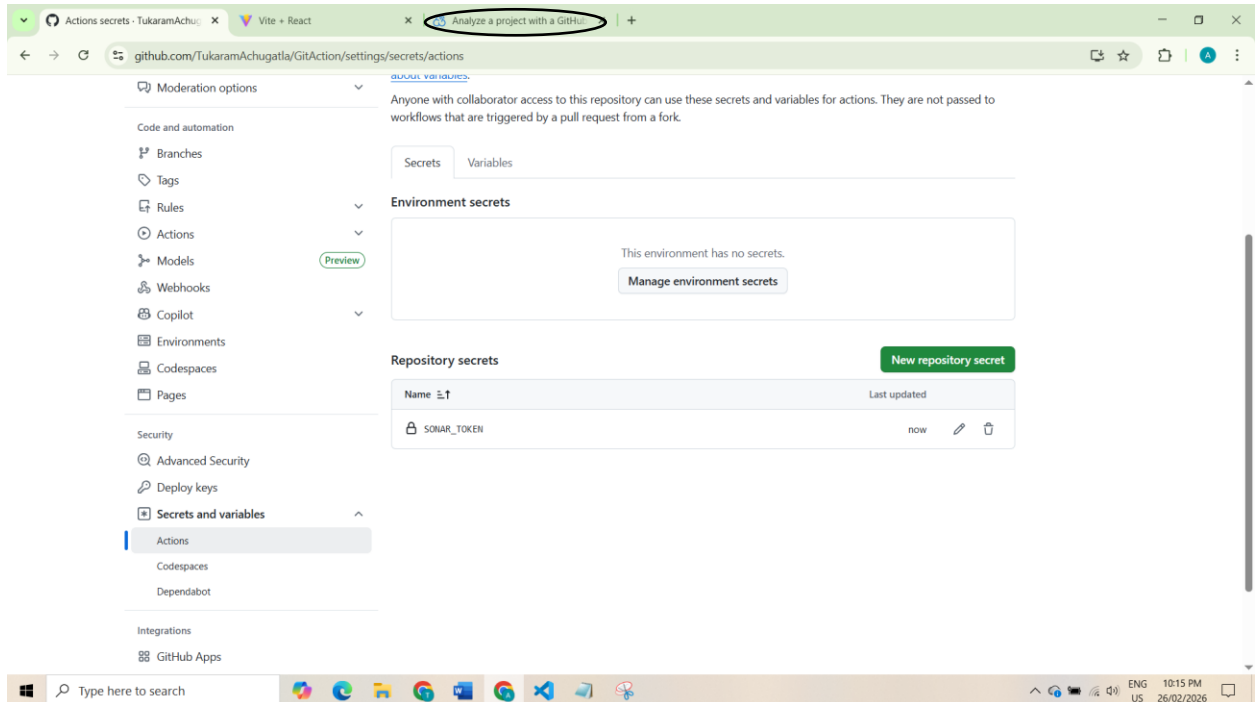
2a263a65c3b64d54d317014d8e165ccd114927ff

Paste the Value field Here From Notepad

Add secret

Click Here

- Go back to the SonarCloud dashboard and scroll down to **Other CI**.



- Copy the provided GitHub Actions YAML code.
- In VS Code, create a new file named `.github/workflows/sonarIssues.yaml`
- Paste the copied YAML code into this file.
- In the SonarCloud dashboard, scroll down and copy the **sonar-project.properties** content.
- Create a new file in your project root (outside `.github/workflows`) named `sonar-project.properties`

- Paste the copied content into this file.

3 Create a `sonar-project.properties` **file**

Create a configuration file in the root directory of the project and name it `sonar-project.properties`

```
sonar.projectKey=TukaramAchugatla_GitAction
sonar.organization=tukaramachugatla

# This is the name and version displayed in the SonarCloud UI.
#sonar.projectName=GitAction
#sonar.projectVersion=1.0

# Path is relative to the sonar-project.properties file. Replace "\" by "/" on Windows.
#sonar.sources=.

# Encoding of the source code. Default is default system encoding
#sonar.sourceEncoding=UTF-8
```

[Copy](#)

- Push the code to GitHub
- Go to the **Actions** tab and verify that the SonarCloud workflow runs successfully.

TukaramAchugatla / GitAction

Code Issues Pull requests **Actions** Projects Wiki Security Insights Settings

← Sonar Cloud Scan

✓ Added Sonar Scan #2

Summary

All jobs

✓ SonarQube

Run details

Usage

Workflow file

Triggered via push 1 minute ago

TukaramAchugatla pushed · ceeeb43 main

Status: Success

Total duration: 44s

Artifacts: -

SonarIssues.yaml

on: push

✓ SonarQube 42s

Create a new branch named “newh5tag”

To <https://github.com/TukaramAchugatla/GitAction.git>

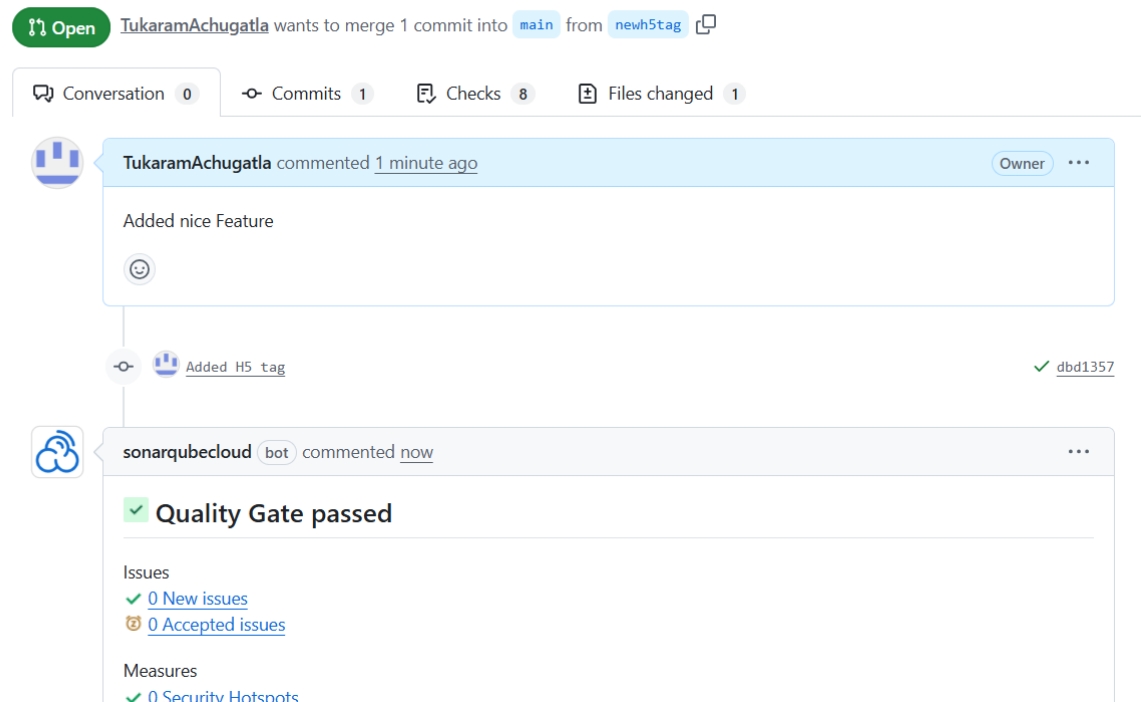
```
e849833..ceecb43 main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\HP\Desktop\Kubernets\Git Action\cicd-github-actions-project> git checkout -b "newh5tag"
Switched to a new branch 'newh5tag'
PS C:\Users\HP\Desktop\Kubernets\Git Action\cicd-github-actions-project> 
```

Add the following tag in App.jsx :<h5>with SonarCloud</h5>

```
<>
  <h1>Github Actions</h1>
  <h2>This is React App</h2>
  <h5>with SonarCloud</h5>
</>
```

- Push the branch to GitHub.
- Raise a **Pull Request (PR)** from `newh5tag` to `main`.
- Wait for the SonarQubeCloud check to complete.
- If the Quality Gate passes, click **Merge**.
- After merging, SonarQubeCloud should pass successfully.

Added H5 tag #1



- Finally, open the **SonarQube Cloud dashboard** to view the complete analysis report

